

Scientific Computation

Prof. Matthew Giamou
Sept. 8, 2025

Lecture 3 - Floating Point Arithmetic

Last Week

- Fixed-point vs. floating-point representations

- Floating-point number systems
- Rounding
- Machine epsilon
- IEEE 754 standard

IEEE 754 - Exceptional Values

$\pm\text{Inf}$: when results *overflow*, e.g. $\frac{1.0}{0.0}$
 $\text{Lower max} = +10^b = \text{Inf}$

NaN : results from undefined operations
 e.g. $\frac{0.0}{0.0}$, $0 \cdot \text{Inf}$, $\frac{\text{Inf}}{\text{Inf}}$

NaN propagate through conditions:

$$\begin{array}{c} f(x_1 + 2) = \text{NaN} \\ \downarrow \\ \text{NaN} \end{array}$$

4

→ Biased exponent: $1 \rightarrow 254$

→ Inf: - sign bit 0 for +Inf and 1 for -Inf

- 2.1.1: $\text{sig} \text{ bit}_i \in \{0, 1\}$

biased exponent all 1s

fractional: at least one of 1

5

Double Precision

Bias value 1023

biased exponent range from -1022 to 1023

Everything else is same

6

Lecture Outline

- Biased exponent
- Floating point arithmetic
- IEEE 754 details: denormalized numbers and the sticky bit

Two's Complement

Recall the usual **two's complement** method of storing signed integers:



-8: 1000

→ extra values (no redundant zero rep.)

→ "underflow" of unsigned 8-bit factor make sense so + and - are the same as unsigned + and -.

Why a biased exponent?

Consider an IEEE floating point number as a signed integer; i.e. interpret the sequence of bits as if they were a signed integer

Easy to compare the magnitude (size, i.e. $=$, $>$) of integers.

What would happen if we stored the exp as a signed integer in 2's complement

Consider for single precision and look at 2 numbers x and y such that $x > y$, and

$\text{exp}(x) = 5$ (unbiased) and $x, y > 0$
 $\text{exp}(y) = -5$

9

Comparisons

x 's exponent in 8 bits: 0 0000 0101 = 5

y 's exponent in 8 bits: 0 1111 1011 = -5

If we compare these as integer values bit by bit:

we'd get that $y > x$

This is why we don't use a signed 2's complement rep. for the exponents. Instead we store an unsigned int for

for the exponent. Another, fast comparison, and arithmetic implement subtract the bias.

10

Lecture Outline

- Biased exponent
- Floating point arithmetic
- IEEE 754 details: denormalized numbers and the sticky bit

FP Arithmetic

For a real number $x \in \mathbb{R}$ and $fl: \mathbb{R} \rightarrow FP$ rounding

$|x| < 1 \Rightarrow \underline{\text{Emach}}$

to nearest, $fl(x) = x(1 \pm \epsilon)$, $|\epsilon| \leq u = \frac{1}{2}$
 ↓
 unit round off

The arithmetic operations are also "correctly" rounded,
 for $x, y \in IEEE\ FP$

$$fl(x \circ y) = (x \circ y)(1 \pm \epsilon), |\epsilon| \leq u$$

$$\circ \in \{+, -, \times, \div\}$$

Also correctly rounded are:

- conversions between formats and strings
- $\sqrt{}$
- fused multiply and add (FMA): compute $ax + b$

12

FP Arithmetic

Example 1: Consider a decimal floating point system with $t = 5$ and
 rounding to nearest $\beta = 10$

→ The machine epsilon (ϵ_{mach}) is $|1.0001 - 1| = 10^{-4}$

→ The unit round off $u = \frac{10^{-4}}{2} = 5 \times 10^{-5}$

→ Let $x = 1.162611735194631$, with rounding to nearest
 we have $fl(x) = 1.1626$

$$fl(x) = x(1 \pm \epsilon) \Rightarrow \epsilon = \frac{fl(x) - x}{x} = 1.0094 \times 10^{-5}$$

$$|\epsilon| = 1.0094 \times 10^{-5} \leq 5 \times 10^{-5} = u$$

FP Arithmetic

Example 2: Assume $t = 5$. Suppose x is close to the middle of two FP numbers, e.g. $x = 1.000050000000000001 \times 10^4$.

$$\xi = \frac{fl(x) - x}{x} = \frac{1.0001 \times 10^{-4} - x}{x} \\ \approx 4.9998 \times 10^{-5}$$

$$\epsilon_{mach} = 10^{-4}, u = 5 \times 10^{-5}$$

$$|\xi| = 4.9998 \times 10^{-5} \leq u$$

ϵ_{mach} and u are fixed parameters for a FP system, (β, t, L, U)

14

FP Arithmetic

Example 3: Assume x, y, z are FP numbers. Find the error in $fl(z(x + y))$.

$$\begin{aligned} fl(z(x+y)) &= fl(z) - fl(x+y) \cdot (1 + \delta_1) \\ &\quad \hookrightarrow \text{round off error in "*"} \\ &= z - fl(x+y) (1 + \delta_1) \\ &= z(x+y) (1 + \delta_1) (1 + \delta_2) \\ &\quad \hookrightarrow \text{ROE, in } fl(x+y) \\ &= z(x+y) (1 + \delta_1 + \delta_2 + \delta_1 \delta_2) \end{aligned}$$

$$= 2^{x+y} (1 + \delta_1 + \delta_2) \quad \rightarrow \text{if } \delta_1 + \delta_2 < 1$$

$$= 2^{x+y} (1 + \delta_1 + \delta_2)$$

$$|\delta_1| \leq u, \quad \delta \triangleq \delta_1 + \delta_2$$

$$|\delta_2| \leq u \quad |\delta| \leq |\delta_1 + \delta_2| \leq |\delta_1| + |\delta_2| \leq 2u$$

15

Lecture Outline

- Biased exponent
- Floating point arithmetic
- **IEEE 754 details: denormalized numbers and the sticky bit**

IEEE 754 Overview

We describe a FP system with (β, t, L, U)

For example, $(\beta = 10, t = 4, L = -2, U = 2)$ gives us numbers like 1.207×10^{-1}

But we've glossed over the particulars of a few important concepts: normalized/denormalized numbers, sticky bit, and the biased exponent

Let's **very carefully** look at the details of how IEEE 754 numbers are represented

17

IEEE 754 Overview

Interpreting the value of a floating point number $x = (\text{sign}, e, d)$ depends on the value of the **biased exponent** bits e .

Case 1 - Normalized Numbers

If $e \neq 0$, we have a **normalized** floating point number:

$$\begin{aligned} x &= (-1)^{\text{sign}} \times \beta^{e-\text{offset}} \times \left(1 + \sum_{i=1}^{t-1} d_i \beta^{-i}\right) \\ &= (-1)^{\text{sign}} \times \beta^{e-\text{offset}} \times (1.d_1 d_2 \cdots d_{t-1}) \end{aligned}$$

IEEE 754 Overview

Interpreting the value of a floating point number $x = (\text{sign}, e, d)$ depends on the value of the **biased exponent** bits e .

Case 2 - Denormalized Numbers

If $e = 0$, we have a **denormalized** floating point number:

$$\begin{aligned} x &= (-1)^{\text{sign}} \times \beta^{e-\text{offset}+1} \times (0 + \sum_{i=1}^{t-1} d_i \beta^{-i}) \\ &= (-1)^{\text{sign}} \times \beta^{e-\text{offset}+1} \times (0.d_1 d_2 \dots d_{t-1}) \end{aligned}$$

The sticky bit (d_0) is “free” because it is always determined by the value of the exponent e !

IEEE 754 Overview

Interpreting the value of a floating point number $x = (\text{sign}, e, d)$ depends on the value of the **exponent** bits e .

Case 3 - Exceptional Values

If the bits of e are all ones and the mantissa (d) is all zeros, then

we have $\pm\text{Inf}$

If the bits of e are all ones and there is at least one nonzero value in the bits of the mantissa, then we have NaN

Finally, if all bits (except sign) are zero, we have ± 0 (some conventions consider zero a denormalized number)

20

其实normalized和denormalized最大的区分就是指数 e 是不是为全0（即十进制的0），即使normalized number也可以表示成最低的指数（比如32位， 1.0×10^{-126} 是完全合法的），但是事实上其指数是1而不是0（1-127），而denormalized number是指数为0，因为此时其指数计算方法是 $e-t+1$ 而不是normalized的 $e-t$ 。

把十进制小数数字转化为FP数的步骤：

1. 先分别把这个数字的整数和小数部分转化成二进制。
2. 然后把这个数字写成 1.010101×2^n 的形式
3. 计算exponent, sign和mantissa bit, 别忘了先计算bias。

小数部分的转化规则如下：

不断的用小数乘以2，每次只取整数部分（0或者1），然后作为0.（小数位）后面的小数位，然后依次重复。

例 3: 0.1

- $0.1 \times 2 = 0.2 \rightarrow$ 整数 0
- $0.2 \times 2 = 0.4 \rightarrow$ 整数 0
- $0.4 \times 2 = 0.8 \rightarrow$ 整数 0
- $0.8 \times 2 = 1.6 \rightarrow$ 整数 1 (小数部分 0.6)
- $0.6 \times 2 = 1.2 \rightarrow$ 整数 1 (小数部分 0.2)
- 接着又回到 0.2, 循环下去

所以：

$$0.1_{10} = 0.0001100110011\dots_2 \quad (\text{无限循环})$$